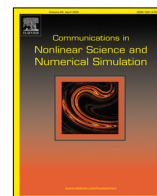




Contents lists available at ScienceDirect

Communications in Nonlinear Science and Numerical Simulation

journal homepage: www.elsevier.com/locate/cnsns

Research paper

Randomized neural network with Petrov–Galerkin methods for solving linear and nonlinear partial differential equations

Yong Shang, Fei Wang^{1,*}, Jingbo Sun

School of Mathematics and Statistics, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, PR China

ARTICLE INFO

Article history:

Received 28 March 2023

Received in revised form 2 August 2023

Accepted 5 September 2023

Available online 11 September 2023

MSC:

65N30

41A46

Keywords:

Petrov–Galerkin method

Randomized neural network

Least-squares method

Space–time approach

ABSTRACT

We present a new approach for solving partial differential equations (PDEs) based on randomized neural networks and the Petrov–Galerkin method, which we call the RNN-PG methods. This method uses randomized neural networks to approximate unknown functions and allows for a flexible choice of test functions, such as finite element basis functions, Legendre or Chebyshev polynomials, or neural networks. We apply the RNN-PG methods to various problems, including Poisson problems with primal or mixed formulations, and time-dependent problems with a space–time approach. This paper is adapted from the work originally posted on [arXiv.com](https://arxiv.org/abs/2201.12995) by the same authors (arXiv:2201.12995, Jan 31, 2022). The new ingredients include non-linear PDEs such as Burger's equation and a numerical example of a high-dimensional heat equation. Numerical experiments show that the RNN-PG methods can achieve high accuracy with a small number of degrees of freedom. Moreover, RNN-PG has several advantages, such as being mesh-free, handling different boundary conditions easily, solving time-dependent problems efficiently, and solving high-dimensional problems quickly. These results demonstrate the great potential of the RNN-PG methods in the field of numerical methods for PDEs.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

Deep neural networks (DNNs) are widely used in artificial intelligence, scientific computing, and other machine learning tasks due to their ability to produce a wide range of nonlinear functions through compositional construction. Their powerful universal approximation capability has made them popular for solving partial differential equations (PDEs). Several DNN-based methods [1–11] have been proposed to overcome the “curse of dimensionality” associated with traditional PDE solvers like the finite element method (FEM). FEM requires discretization of the domain of interest, and as the problem dimension increases, the number of mesh points also increases exponentially, making FEM computationally intractable. In addition, mesh generation is time-consuming in many cases.

One approach among these DNN-based methods is to establish the algorithm using the strong form and minimize the residual of the original PDEs, such as physical-informed neural networks (PINNs), deep Galerkin method (DGM), mixed residual method (MIM), and so on. PINNs, introduced in [6], use neural networks to approximate the PDE solution by minimizing the mean squared errors of residual terms, such as observed data, differential equations, boundary conditions, and/or initial conditions. Other variants, such as fractional PINNs and nonlocal PINNs, have been proposed in [12–15].

* Corresponding author.

E-mail addresses: fsy2503@stu.xjtu.edu.cn (Y. Shang), feiwang.xjtu@xjtu.edu.cn (F. Wang), jingbosun.xjtu@xjtu.edu.cn (J. Sun).

¹ The work of this author was partially supported by the National Natural Science Foundation of China (Grant No. 12171383).

DGM formulates the loss function as the PDE residual in the least-squares sense to measure the degree of satisfaction of the approximate solution to the differential operator and boundary conditions [16]. MIM rewrites the PDE into a lower-order system and uses the system residual as the loss function [17]. Recently, a least-squares neural network method was proposed in [18] for solving the advection-reaction problem with a discontinuous solution, and a deep least-squares method based on the least-squares functional of the first-order system was studied in [19].

Another approach utilizes the weak formulations of PDEs. The deep Ritz Method uses the variational formulation of symmetric PDEs and the integral of the boundary residual as the loss function, and the DNNs are then trained by minimizing the numerical quadrature of the loss function [5]. The deep Nitsche Method [20] adopts Nitsche's formulation as the loss function to handle essential boundary conditions. Based on the Euler-Lagrange equation of the energy functional, a penalty-free neural network method [21] is proposed to solve second-order boundary-value problems. Weak adversarial networks [22] reformulate the PDEs as saddle-point problems in the weak formulation and approximate trial and test functions by two neural networks. They are trained alternately as a minimization problem under the operator norm. Various neural networks based on the Petrov-Galerkin scheme have been developed with different choices for trial functions and test spaces. For instance, variational physics-informed neural networks (VPINN) use neural networks as their trial function space and Legendre polynomials as their test space [23]. Variational neural networks [24] employ piecewise polynomials as their test functions, and *hp*-VPINNs [25] improve accuracy by selecting non-overlapping polynomials as test functions on each subdomain. The convergence rate of VPINN with piecewise polynomial test functions was analyzed in [26].

These DNN-based methods have made significant advancements in solving high-dimensional and irregular domain problems, and incorporating observed data into the loss function makes them suitable for handling inverse problems. However, limitations in accuracy and training time remain a challenge. Optimizers used for training have limitations [13], even though DNNs have excellent approximation properties. Additionally, the training process can be computationally expensive, taking hours to reach the desired accuracy, whereas traditional methods like FEM take only a few seconds. Therefore, it is necessary for further research to improve the efficiency of DNN-based methods for solving general PDEs. By combining domain decomposition and extreme learning machines (ELM), the local ELM approach solves the neural network parameters with least-squares computations, achieving high-precision results without using an optimizer for training [27]. It enforces certain continuity conditions on interior edges to connect different neural networks on neighboring subdomains. To relax the C^1 continuity condition in [27], overlapping domain decomposition was employed in [28]. Moreover, [29] proposed a hybrid of least-squares methods and gradient descent training algorithm, which can speed up the training for regression problems and PINNs.

In this paper, based on the variational principle, we propose Randomized Neural Network with Petrov-Galerkin Methods (RNN-PG methods), which approximates numerical solutions using randomized NNs and employs different approaches for choosing test functions. The neural network parameters are given randomly and fixed, except for the last layer, reducing the number of parameters to be trained and facilitating the assembling of the linear system, which improves accuracy and reduces the computational cost. Additionally, the RNN-PG method uses a space-time approach to jointly treat temporal and spatial variables, taking initial conditions as boundary conditions, allowing it to solve time-dependent problems without iteration steps. The neural networks produced by the RNN-PG method can provide the numerical solution value at any given space-time point without interpolation.

The paper is organized as follows. Section 2 introduces the basic ideas of the RNN-PG method for solving elliptic PDEs. Section 3 establishes mixed RNN-PG methods for the Poisson problem with four mixed formulations. Section 4 discusses solving linear and nonlinear time-dependent problems by the RNN-PG method in the space-time approach. Section 5 presents numerical examples to show the efficiency and accuracy of RNN-PG methods. The last section concludes the paper with discussions.

2. RNN-PG method

In this section, we introduce the basic concepts of the randomized neural network with the Petrov-Galerkin method and demonstrate its application to solve an elliptic partial differential equation.

Consider a partial differential equation

$$\mathcal{A}u = f \quad \text{in } \Omega, \quad (2.1)$$

$$\mathcal{B}u = g \quad \text{on } \Gamma, \quad (2.2)$$

where Ω is a bounded domain in $\mathbb{R}^d$ with boundary $\Gamma = \Gamma_D \cup \Gamma_N$ and $\Gamma_D \cap \Gamma_N = \emptyset$. For example, when the differential operator $\mathcal{A}$ is given by

$$\mathcal{A} := -\nabla \cdot (\alpha(\mathbf{x})\nabla) + \delta(\mathbf{x}), \quad (2.3)$$

where $\alpha_1 > \alpha(\mathbf{x}) \geq \alpha_0 > 0$, $\delta_1 \geq \delta(\mathbf{x}) \geq \delta_0 > 0$, (2.1) is a diffusion-reaction equation. Here, α_0 , α_1 , δ_0 and δ_1 are constants. The boundary condition (2.2) can be Dirichlet, Neumann, or Robin types. In this paper, we consider the following boundary conditions:

$$u = g_D \quad \text{on } \Gamma_D, \quad (2.4)$$

$$\alpha(\mathbf{x})\nabla u \cdot \mathbf{n} = g_N \quad \text{on } \Gamma_N. \quad (2.5)$$

2.1. Petrov–Galerkin method

Under proper conditions, the PDE (2.1) with boundary conditions (2.4)–(2.5) has the weak formulation: Find $u \in U_{D,g_D} = \{u \in U; u|_{\Gamma_D} = g_D\}$ such that

$$a(u, v) = l(v) \quad \forall v \in V_{D,0} = \{v \in V; v|_{\Gamma_D} = 0\}, \quad (2.6)$$

where U and V are two Hilbert spaces with inner products $(\cdot, \cdot)_U$ and $(\cdot, \cdot)_V$, as well as corresponding norms $\|\cdot\|_U$ and $\|\cdot\|_V$, respectively. Here, $a(\cdot, \cdot) : U \times V \rightarrow \mathbb{R}$ is a continuous bilinear form with

$$a(u, v) \leq M \|u\|_U \|v\|_V \quad \forall u \in U, v \in V \quad (2.7)$$

for a constant $M > 0$. By Babuška Theorem [30,31], the problem (2.6) has a unique solution if and only if the following conditions hold:

$$\inf_{u \in U} \sup_{v \in V} \frac{a(u, v)}{\|u\|_U \|v\|_V} = \inf_{v \in V} \sup_{u \in U} \frac{a(u, v)}{\|u\|_U \|v\|_V} = \beta > 0, \quad (2.8)$$

which is known as the Babuška–Brezzi or inf-sup condition [32]. If we choose $\mathcal{A}$ as (2.3), the bilinear form and linear form in (2.6) are given by

$$\begin{aligned} a(u, v) &= \int_{\Omega} (\alpha(\mathbf{x}) \nabla u \cdot \nabla v + \delta(\mathbf{x}) u v) \, d\mathbf{x}, \\ l(v) &= \int_{\Omega} f v \, d\mathbf{x} + \int_{\Gamma_N} g_N v \, ds, \end{aligned}$$

and the bilinear form $a(u, v)$ satisfies the conditions (2.7) and (2.8).

To numerically solve the problem (2.6), one can apply the Petrov–Galerkin (PG) method: Find $u_h \in U_h \subset U_{D,g_D}$ such that

$$a(u_h, v_h) = l(v_h) \quad \forall v_h \in V_h \subset V_{D,0}. \quad (2.9)$$

Here, U_h and V_h are finite-dimensional function spaces and usually consist of piecewise polynomial functions. By (2.8), we know that the problem (2.9) is well-posed if and only if the following conditions hold

$$\inf_{u_h \in U_h} \sup_{v_h \in V_h} \frac{a(u_h, v_h)}{\|u_h\|_U \|v_h\|_V} = \inf_{v_h \in V_h} \sup_{u_h \in U_h} \frac{a(u_h, v_h)}{\|u_h\|_U \|v_h\|_V} = \beta_h > 0. \quad (2.10)$$

Furthermore, when U_h and V_h are finite-dimensional, the above two conditions are reduced to one [33].

Remark 2.1. The PG method requires careful design of the finite element pair U_h and V_h and verification of the inf-sup condition (2.10). These tasks can be challenging and technical for general users. The inf-sup condition ensures the invertibility of the resulting matrix A from $a(u_h, v_h)$ of the PG method, which allows the linear system to be solved by direct methods or iterative methods. Our approach does not need to verify the inf-sup condition for the trial and test function spaces, because we use the least-square method to solve the linear system, which does not require A to be invertible.

2.2. Neural networks

There are a variety of neural network structures, let us introduce the neural networks used in this paper as follows. Let $D, n_i \in \mathbb{N}^+$, and n_i is the number of neurons in the i th layer, ρ is the activation function. A fully connected neural network with the depth of D is a function $\Phi : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_D}$ defined by

$$\begin{aligned} \Phi_0(\mathbf{x}) &= \mathbf{x}, \\ \Phi_l(\mathbf{x}) &= \rho(\mathbf{W}_l \Phi_{l-1} + \mathbf{b}_l) \quad \text{for } l = 1, \dots, D-1, \\ \Phi &:= \Phi_D(\mathbf{x}) = \mathbf{W}_D \Phi_{D-1} + \mathbf{b}_D, \end{aligned} \quad (2.11)$$

where $\mathbf{W}_l = (w_{ij}^{(l)}) \in \mathbb{R}^{n_l \times n_{l-1}}$, $\mathbf{b}_l = (b_i^{(l)}) \in \mathbb{R}^{n_l}$, and $\phi_l = \{\mathbf{W}_l, \mathbf{b}_l\}$ is the set of weight parameters in the l th layer. In this paper, we set $n_0 = d$ for stationary problems and $n_0 = d + 1$ for time-dependent problems.

In addition, we also adopt the ResNet [34], in which the input layer is a fully-connected layer with the number of neurons $n_0 = d$, each layer of this network is constructed by stacking several blocks, each block consists of two linear transformations, two activation functions, and a residual connection. Thus a ResNet Ψ with D layers takes the form as:

$$\begin{aligned} \Psi_0(\mathbf{x}) &= \rho(\mathbf{W}_0 \mathbf{x} + \mathbf{b}_0), \\ \Psi_l(\mathbf{x}) &= \rho(\mathbf{W}_{l,2} \rho(\mathbf{W}_{l,1} \Psi_{l-1} + \mathbf{b}_{l,1}) + \mathbf{b}_{l,2}) + \Psi_{l-1} \quad \text{for } l = 1, \dots, D-1, \end{aligned}$$

$$\Psi := \Psi_D(\mathbf{x}) = \mathbf{W}_D \Psi_{D-1} + \mathbf{b}_D,$$

where $\{\mathbf{W}_{l,1}, \mathbf{b}_{l,1}\}$ and $\{\mathbf{W}_{l,2}, \mathbf{b}_{l,2}\}$ are the weight parameters of the first and second linear transformation in the l th layer, respectively.

Denote the number of nonzero entries of a matrix $\mathbf{W}_k$ by

$$\|\mathbf{W}_k\|_{l^0} := \left| \{(i, j) : w_{ij}^{(k)} \neq 0\} \right|,$$

then

$$N_D := \sum_{j=1}^D (\|\mathbf{W}_j\|_{l^0} + \|\mathbf{b}_j\|_{l^0})$$

is the total number of nonzero weights of the neural network Φ . For two positive constant integers M_D and B_D , we give a class of neural network functions as follows

$$\mathcal{N}_\rho(D, M_D, B_D) := \{\Phi \text{ defined by (2.11) with depth } D, N_D \leq M_D \text{ and } |w_{ij}^{(l)}| \leq B_D, |b_i^{(l)}| \leq B_D\}.$$

The universal approximation theorem [35,36] states that shallow neural networks with continuous, non-polynomial activation functions can uniformly approximate every continuous function on a compact domain. The relationship between ReLU-DNN and linear finite element function was studied in [4]. Further results were established in [37–39] for activation functions with a certain regularity, with approximation errors given in the sense of L^p norm. Error bounds for Hölder functions and functions in the Sobolev space $W^{n,\infty}$ were given under L^∞ norm in [40,41]. Error bounds for ReLU and ELU neural networks were derived in Sobolev norms in [42], which apply to many practical activation functions such as logistic, tanh, arctan, and others, providing a useful tool for error analysis in DNN-based methods.

Theorem 2.2 (Proposition 4.8 in [42], Theorem 4.1 in [43]). *Given $p \geq 1$, $s, k, d \in \mathbb{N}^+$, $s \geq k + 1$. Let ρ be the logistic function $\frac{1}{1+e^{-x}}$ or tanh function $\frac{e^x - e^{-x}}{e^x + e^{-x}}$. Denote $\mathcal{F}_{s,p,d} := \{f \in W^{s,p}([0, 1]^d) : \|f\|_{W^{s,p}([0, 1]^d)} \leq 1\}$. For any $\epsilon > 0$ and $f \in \mathcal{F}_{s,p,d}$, there exists a neural network $f_\rho \in \mathcal{N}_\rho(D, M_D, B_D)$ with depth $D \leq C \log(d + s)$, $M_D \leq C \cdot \epsilon^{-d/s-k-\mu k}$, and $B_D \leq C \cdot \epsilon^{-\theta}$ such that*

$$\|f - f_\rho\|_{W^{k,p}([0, 1]^d)} \leq \epsilon.$$

where C, θ are constants depending on d, s, p, k ; μ is an arbitrarily small positive number.

Remark 2.3. The bounds provided in the above theorem can be found in Proposition 4.8 in [42], and the bound on the depth is explicitly given in Theorem 4.1 in [43]. Based on these findings, we know that PDE solutions can be efficiently approximated by neural networks with an adequate number of layers and nonzero weights.

2.3. RNN-PG scheme

In this paper, we propose a numerical scheme to approximate the exact solution $u(\mathbf{x})$ using a neural network $u_\rho(\mathbf{x}; \theta)$ with proper parameters θ . The scheme, named Randomized Neural Network with Petrov–Galerkin (RNN-PG) method, aims to find $u_\rho \in U_\rho \subset \mathcal{N}_\rho(D, M_D, B_D) \subset U$ such that

$$a(u_\rho, v_h) = l(v_h) \quad \forall v_h \in V_h \subset V_{D,0}, \quad (2.12)$$

$$u_\rho(\mathbf{x}_k) = g_D(\mathbf{x}_k) \quad \text{for some points } \mathbf{x}_k \in \Gamma_D, \quad k = 1, 2, \dots, N_b, \quad (2.13)$$

where U_ρ is the trial function space, which refers to a class of neural network functions with the activation function ρ and a specific structure introduced in the previous subsection. The test function space V_h is chosen as a finite-dimensional function space that is a good approximation of $V_{D,0}$, such as finite element spaces. In this paper, we mainly use bilinear finite element spaces for 2D problems and trilinear finite element spaces for 3D problems. Without loss of generality, we assume that $\{v_j\}_{j=1}^G$ is a group of functions belonging to $V_{D,0}$ and $\bigcup_{j=1}^G \text{supp}\{v_j\} = \Omega$, then we choose $V_h := \text{span}\{v_j\}_{j=1}^G$.

Since $\mathcal{N}_\rho(D, M_D, B_D)$ involves a nonlinear activation function, it is generally not a function space because it may not be closed under linear combinations. That is, $\alpha u + \beta v$ may not be in $\mathcal{N}_\rho$ for some $u, v \in \mathcal{N}_\rho$ and scalars α, β . So we do not choose it as U_ρ , instead, we choose a function space, a subset of $\mathcal{N}_\rho(D, M_D, B_D)$ as U_ρ , which is constructed as follows and showed in Fig. 1. For any $u_\rho \in U_\rho \subset \mathcal{N}_\rho(D, M_D, B_D)$, we choose the initial weights ϕ_l with values drawn from the uniform distribution $\mathcal{U}(-r, r)$ for each layer and fixed them except for the last layer, where $r \in \mathbb{R}$. Let Φ_{D-1}^j denote the output of the j th neuron in the $D - 1$ layer ($j = 1, 2, \dots, n_{D-1}$), $\mathbf{W}_D = \left((w_{ij}^{(D)}) \right) \in \mathbb{R}^{n_D \times n_{D-1}}$ with $n_D = 1$, and $\mathbf{b}_D = \mathbf{0}$. Then the output of the neural network

$$\Phi := \Phi_D(\mathbf{x}) = \mathbf{W}_D \Phi_{D-1} + \mathbf{b}_D = \sum_{j=1}^{n_{D-1}} w_{1j}^{(D)} \Phi_{D-1}^j(\mathbf{x}).$$

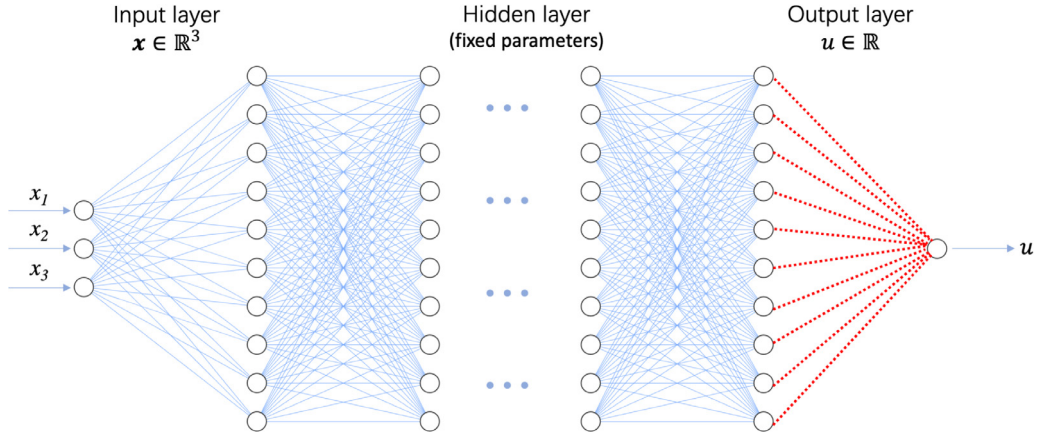


Fig. 1. Network structure of $u_\rho : \mathbb{R}^3 \rightarrow \mathbb{R}$, the solid blue lines represent the parameters of the neural network that are randomly initialized and fixed thereafter, while the dotted red lines refer to the parameters that need to be solved.

For simplicity, we denote $\Phi_j^u := \Phi_{D-1}^j$ and $u_\rho^j := w_{ij}^D$ for $j = 1, \dots, n_{D-1}$. Then

$$u_\rho(\mathbf{x}) = \sum_{j=1}^{n_{D-1}} u_\rho^j \Phi_j^u(\mathbf{x}). \quad (2.14)$$

Notice that the values of the initial weights ϕ_l ($l = 1, \dots, n_{D-1}$) are given randomly and fixed, so these weights do not need to be trained or solved, then $\{\Phi_j^u(\mathbf{x})\}_{j=1}^{n_{D-1}}$ can be regarded as a group of basis functions. Hence,

$$U_\rho = \text{span}\{\Phi_1^u(\mathbf{x}), \dots, \Phi_{n_{D-1}}^u(\mathbf{x})\}.$$

We see that the degrees of freedom for the RNN-PG method is the number of neurons in the $D - 1$ layer, that is, n_{D-1} .

Remark 2.4. The neural network in Fig. 1 is a randomized neural network, following the framework introduced in [44,45]. An example of such a network is the extreme learning machine [46]. In [47], it was demonstrated that ELM has similar generalization capabilities as traditional neural networks if appropriate activation functions and initialization methods for fixed parameters are used. A similar approach of solving the parameters with red lines in Fig. 1 using a least-squares method was proposed in [29], but they also update the parameters with blue lines using gradient descent and then iterate.

Then, the problem (2.12) becomes: Find $u_\rho^j, j = 1, \dots, n_{D-1}$, such that

$$\sum_{j=1}^{n_{D-1}} u_\rho^j a(\Phi_j^u(\mathbf{x}), v_i) = l(v_i) \quad \forall v_i \in V_h \subset V_{D,0}. \quad (2.15)$$

The above equation gives

$$\mathbf{A}\mathbf{U} = \mathbf{L},$$

where $\mathbf{A}_{i,j} = a(\Phi_j^u(\mathbf{x}), v_i)$, $\mathbf{U} = (u_\rho^1, \dots, u_\rho^{n_{D-1}})^T$, and $\mathbf{L} = (l_1, \dots, l_{N_h})^T$ with $l_i = l(v_i)$ ($i = 1, \dots, N_h$). Note that the Neumann boundary condition (2.5) is included in the weak formulation. However, we need to enforce the Dirichlet boundary condition (2.13). To do so, we can take some random samples $\{\mathbf{x}_k\}_{k=1}^{N_b}$ according to the uniform distribution $\mathcal{U}(\Gamma_D)$, and set $u_\rho(\mathbf{x}_k) = g_D(\mathbf{x}_k)$ for all $\mathbf{x}_k \in \Gamma_D$, i.e.,

$$\sum_{j=1}^{n_{D-1}} u_\rho^j \Phi_j^u(\mathbf{x}_k) = g_D(\mathbf{x}_k) \quad \text{for } k = 1, \dots, N_b. \quad (2.16)$$

Similarly, the above equation implies $\mathbf{B}\mathbf{U} = \mathbf{G}$, where $\mathbf{B}_{k,j} = \Phi_j^u(\mathbf{x}_k)$ and $\mathbf{G} = (g_1, \dots, g_{N_b})^T$ with $g_k = g_D(\mathbf{x}_k)$. Thus, we can obtain the solution u_ρ by solving a least-squares problem with the linear system

$$\begin{bmatrix} \mathbf{A} \\ \mathbf{B} \end{bmatrix} \mathbf{U} = \begin{bmatrix} \mathbf{L} \\ \mathbf{G} \end{bmatrix}. \quad (2.17)$$

The algorithm for the RNN-PG method is summarized as follows.

Algorithm 1 Randomized Neural Network with Petrov-Galerkin Method

- Step 1. Initialize network architecture $u_\rho : \Omega \rightarrow \mathbb{R}$ with depth D and parameters ϕ_l for $l = 1, \dots, D$.
 Step 2. Fix the parameters ϕ_l for $l = 1, \dots, D - 1$ of u_ρ and rewrite u_ρ as Eq. (2.14).
 Step 3. Choose $v_i \in V_h$, $i = 1, \dots, N_h$ and assemble the linear system $\mathbf{A}\mathbf{U} = \mathbf{L}$ w.r.t. Eq. (2.15).
 Step 4. Take random samples $\{\mathbf{x}_k\}_{k=1}^{N_b}$ according to $\mathcal{U}(\Gamma_D)$ and obtain $\mathbf{B}\mathbf{U} = \mathbf{G}$ w.r.t. Eq. (2.16).
 Step 5. obtain $\mathbf{U}$ by solving the least-squares problem w.r.t. Eq. (2.17).
 Step 6. Update the network parameters ϕ_D of u_ρ .
-

Remark 2.5. We know that the least-squares problem with the linear system (2.17) always has a solution. The error between the neural network solution u_ρ and the exact solution u is dependent on the approximation ability of the function space U_ρ , the choice of test function space V_h , and the collocation points on the Dirichlet boundary condition. Generally, the more suitable test functions we use, and the more collocation points we choose, the better u_ρ we will obtain. However, the accuracy of the RNN-PG method is primarily determined by the distance between the exact solution and the trial function space U_ρ .

Remark 2.6. Most DNN-based numerical methods solve optimization problems to train neural networks, even if the original problem is a linear PDE, which incurs high computation time and often yields suboptimal results due to the difficulty of finding good solvers. To overcome this issue and leverage powerful neural networks to approximate PDE solutions, the ELM approach can be adopted, reducing the problem to solving a linear system. However, as the matrix $\mathbf{A}$ is not square and symmetric, traditional linear solvers cannot be used. Moreover, incorporating essential boundary conditions into the network structure can be challenging, and penalty methods may introduce unexpected errors. To address these issues, the hybrid of ELM and least-squares methods in the Petrov-Galerkin method can yield an efficient numerical method, as demonstrated by numerical experiments.

3. Mixed RNN-PG methods

Similar to the mixed finite element method [48], we can construct Mixed RNN-PG (M-RNN-PG) methods by approximating multiple unknown functions using one or more neural networks simultaneously.

Take the Poisson equation with the mixed boundary conditions as an example

$$\begin{cases} -\nabla \cdot (\nabla u) = f & \text{in } \Omega, & \text{(a)} \\ u = g_D & \text{on } \Gamma_D, & \text{(b)} \\ \nabla u \cdot \mathbf{n} = g_N & \text{on } \Gamma_N. & \text{(c)} \end{cases} \quad (3.18)$$

Set $\mathbf{p} = \nabla u$, we have the following first-order system

$$\begin{cases} \mathbf{p} - \nabla u = 0 & \text{in } \Omega, & \text{(a)} \\ -\nabla \cdot \mathbf{p} = f & \text{in } \Omega, & \text{(b)} \\ u = g_D & \text{on } \Gamma_D, & \text{(c)} \\ \mathbf{p} \cdot \mathbf{n} = g_N & \text{on } \Gamma_N. & \text{(d)} \end{cases} \quad (3.19)$$

On both sides of the Eqs. (3.19)(a)–(3.19)(b), multiplying proper test functions, we can obtain the following mixed formulations by integration by parts and the boundary conditions (3.19)(c)–(3.19)(d).

Mixed Formulation 1: Find $(\mathbf{p}, u) \in (L^2(\Omega))^d \times H_{D,g_D}^1(\Omega)$ such that

$$\begin{aligned} \int_{\Omega} \mathbf{p} \cdot \mathbf{q} \, d\mathbf{x} - \int_{\Omega} \nabla u \cdot \mathbf{q} \, d\mathbf{x} &= 0 & \forall \mathbf{q} \in (L^2(\Omega))^d, \\ \int_{\Omega} \mathbf{p} \cdot \nabla v \, d\mathbf{x} &= \int_{\Omega} f v \, d\mathbf{x} + \int_{\Gamma_N} g_N v \, ds & \forall v \in H_{D,0}^1(\Omega). \end{aligned}$$

Mixed Formulation 2: Find $(\mathbf{p}, u) \in \mathbf{H}_{N,g_N}(\text{div}, \Omega) \times L^2(\Omega)$ such that

$$\begin{aligned} \int_{\Omega} \mathbf{p} \cdot \mathbf{q} \, d\mathbf{x} + \int_{\Omega} u \nabla \cdot \mathbf{q} \, d\mathbf{x} &= \int_{\Gamma_D} g_D \mathbf{q} \cdot \mathbf{n} \, ds & \forall \mathbf{q} \in \mathbf{H}_{N,0}(\text{div}, \Omega), \\ - \int_{\Omega} \nabla \cdot \mathbf{p} v \, d\mathbf{x} &= \int_{\Omega} f v \, d\mathbf{x} & \forall v \in L^2(\Omega). \end{aligned}$$

Mixed Formulation 3: Find $(\mathbf{p}, u) \in \mathbf{H}_{N, \mathbf{g}_N}(\text{div}, \Omega) \times H_{D, \mathbf{g}_D}^1(\Omega)$ such that

$$\begin{aligned} \int_{\Omega} \mathbf{p} \cdot \mathbf{q} \, d\mathbf{x} - \int_{\Omega} \nabla u \cdot \mathbf{q} \, d\mathbf{x} &= 0 \quad \forall \mathbf{q} \in (L^2(\Omega))^d, \\ - \int_{\Omega} \nabla \cdot \mathbf{p} v \, d\mathbf{x} &= \int_{\Omega} f v \, d\mathbf{x} \quad \forall v \in L^2(\Omega). \end{aligned}$$

Mixed Formulation 4: Find $(\mathbf{p}, u) \in (L^2(\Omega))^d \times L^2(\Omega)$ such that

$$\begin{aligned} \int_{\Omega} \mathbf{p} \cdot \mathbf{q} \, d\mathbf{x} + \int_{\Omega} u \nabla \cdot \mathbf{q} \, d\mathbf{x} &= \int_{\Gamma_D} \mathbf{g}_D \cdot \mathbf{n} \, ds \quad \forall \mathbf{q} \in \mathbf{H}_{N,0}(\text{div}, \Omega), \\ \int_{\Omega} \mathbf{p} \cdot \nabla v \, d\mathbf{x} &= \int_{\Omega} f v \, d\mathbf{x} + \int_{\Gamma_N} \mathbf{g}_N v \, ds \quad \forall v \in H_{D,0}^1(\Omega). \end{aligned}$$

Here,

$$\begin{aligned} H_{D,0}^1(\Omega) &= \{v \in H^1(\Omega); v|_{\Gamma_D} = 0\}, \\ H_{D,\mathbf{g}_D}^1(\Omega) &= \{v \in H^1(\Omega); v|_{\Gamma_D} = \mathbf{g}_D\}, \\ \mathbf{H}_{N,0}(\text{div}, \Omega) &= \{\mathbf{q} \in \mathbf{H}(\text{div}, \Omega); \langle \mathbf{q} \cdot \mathbf{n}, v \rangle = 0 \, \forall v \in H_{D,0}^1(\Omega)\}, \\ \mathbf{H}_{N,\mathbf{g}_N}(\text{div}, \Omega) &= \{\mathbf{q} \in \mathbf{H}(\text{div}, \Omega); \langle \mathbf{q} \cdot \mathbf{n}, v \rangle = \mathbf{g}_N \, \forall v \in H_{D,0}^1(\Omega)\}, \end{aligned}$$

and $\langle \cdot, \cdot \rangle$ denotes the duality between $H^{-1/2}(\Gamma)$ and $H^{1/2}(\Gamma)$.

Under appropriate regularity assumptions, the mixed formulations described above are equivalent and can be used to construct the mixed RNN-PG methods. Mixed Formulation 4 has the advantage of having boundary conditions naturally built-in, and does not require the calculation of derivatives for u and $\mathbf{p}$, making it well-suited for implementing the M-RNN-PG method. Now, let us consider Mixed Formulation 4 to implement the M-RNN-PG method, and other mixed formulations can be implemented similarly.

The Mixed Formulation 4 can be rewritten as: Find $(\mathbf{p}, u) \in (L^2(\Omega))^d \times L^2(\Omega)$ such that

$$\mathcal{L}((\mathbf{p}, u); (\mathbf{q}, v)) = \mathcal{F}(\mathbf{q}, v) \quad \forall (\mathbf{q}, v) \in \mathbf{H}_{N,0}(\text{div}, \Omega) \times H_{D,0}^1(\Omega). \quad (3.20)$$

where

$$\begin{aligned} \mathcal{L}((\mathbf{p}, u); (\mathbf{q}, v)) &= \int_{\Omega} (\mathbf{p} \cdot \mathbf{q} + u \nabla \cdot \mathbf{q} + \mathbf{p} \cdot \nabla v) \, d\mathbf{x}, \\ \mathcal{F}(\mathbf{q}, v) &= \int_{\Omega} f v \, d\mathbf{x} + \int_{\Gamma_N} \mathbf{g}_N v \, ds + \int_{\Gamma_D} \mathbf{g}_D \cdot \mathbf{n} \, ds. \end{aligned}$$

Similar to the RNN-PG method introduced in Section 2, we need to find two neural networks to approximate the variables u and $\mathbf{p}$, separately. Therefore, the M-RNN-PG method is: Find neural networks $u_{\rho} \in U_{\rho} \subset \mathcal{N}_{\rho}(D, M_D, B_D)$ and $\mathbf{p}_{\rho} \in \mathbf{Q}_{\rho} \subset \mathcal{N}_{\rho}(\bar{D}, N_{\bar{D}}, B_{\bar{D}})$ such that

$$\mathcal{L}((\mathbf{p}_{\rho}, u_{\rho}); (\mathbf{q}_i, v_k)) = \mathcal{F}(\mathbf{q}_i, v_k) \quad \forall (\mathbf{q}_i, v_k) \in \mathbf{Q}_h \times V_h, \quad (3.21)$$

where $\mathbf{Q}_h$ and V_h can be chosen as any proper finite-dimensional function spaces. We construct $u_{\rho} \in U_{\rho}$ the same as the one in (2.14), i.e.,

$$u_{\rho}(\mathbf{x}) = \sum_{j=1}^{n_{D-1}} u_{\rho}^j \Phi_j^u(\mathbf{x}).$$

Let us construct $\mathbf{p}_{\rho}$ slightly different from u_{ρ} . Specifically, the number of neurons in the last layer for $\mathbf{p}_{\rho}$ is $n_{\bar{D}} = d$. Here, let us consider 2-dimensional case, i.e., $n_{\bar{D}} = 2$, then the output of the neural network for $\mathbf{p}_{\rho}$ is

$$\Phi^p := \Phi_{\bar{D}}^p(\mathbf{x}) = \mathbf{W}_{\bar{D}} \Phi_{\bar{D}-1}^p = \begin{bmatrix} \sum_{j=1}^{n_{\bar{D}-1}} p_{1\rho}^j \Phi_j^p(\mathbf{x}) \\ \sum_{j=1}^{n_{\bar{D}-1}} p_{2\rho}^j \Phi_j^p(\mathbf{x}) \end{bmatrix}.$$

Here, we denote w_{1j}^D and w_{2j}^D by $p_{1\rho}^j$ and $p_{2\rho}^j$, respectively. Therefore, the problem (3.21) becomes: Find u_ρ^j with $j = 1, \dots, n_{D-1}$, and $p_{1\rho}^j, p_{2\rho}^j$ with $j = 1, \dots, n_{D-1}$ such that

$$\mathcal{L} \left(\left(\left[\sum_{j=1}^{n_{D-1}} p_{1\rho}^j \Phi_j^p(\mathbf{x}), \sum_{j=1}^{n_{D-1}} p_{2\rho}^j \Phi_j^p(\mathbf{x}) \right]^T, \sum_{j=1}^{n_{D-1}} u_\rho^j \Phi_j^u(\mathbf{x}) \right); (\mathbf{q}_i, v_k) \right) = \mathcal{F}(\mathbf{q}_i, v_k) \quad \forall (\mathbf{q}_i, v_k) \in \mathbf{Q}_h \times V_h. \quad (3.22)$$

Denote $\mathbf{q}_i = [q_i^1, q_i^2]^T$, and we can take $(\mathbf{q}_i, v_k) \in \mathbf{Q}_h \times V_h$ in the forms of $([q_i^1, 0], 0)$, $([0, q_i^2], 0)$, $([0, 0], v_k)$, separately. Of course, other forms of test functions can be used as well. Finally, we need to solve a least-squares problem with the linear system generated by (3.22).

Remark 3.1. It is important to note that different neural networks provide different sets of bases. If multiple unknown functions need to be approximated in the same function space, a single neural network can be used to approximate all the unknown functions simultaneously, but the number of neurons in the last layer must be adjusted accordingly.

Remark 3.2. In contrast to mixed finite element methods, where the finite element pair $\mathbf{Q}_h \times V_h$ must be chosen carefully to ensure the well-posedness of the discrete problem, M-RNN-PG methods do not need to consider the inf-sup condition due to the use of the least-squares method, making it more flexible in the choice of $\mathbf{Q}_h \times V_h$.

4. Space-time RNN-PG method for time-dependent problems

In this section, we extend the ideas of the RNN-PG method to solve linear and nonlinear time-dependent PDEs using a space-time approach. Rather than discretizing time and space separately, we apply the RNN-PG method to solve time-dependent problems in a unified manner.

Let us consider the following time-dependent problem

$$\mathcal{A}u = f \quad \text{in } \Omega \times I, \quad (4.23)$$

$$\mathcal{B}u = g \quad \text{on } \Gamma \times I, \quad (4.24)$$

$$\mathcal{C}u = h \quad \text{in } \Omega \times \{0\}, \quad (4.25)$$

where $\Omega \subset \mathbb{R}^d$ is a bounded domain with $\Gamma = \Gamma_D \cup \Gamma_N$ and $\Gamma_D \cap \Gamma_N = \emptyset$, and $I = (0, T)$ is the time interval of interest. For example, when the differential operator $\mathcal{A}$ is given by

$$\mathcal{A} := \frac{\partial}{\partial t} - \nabla \cdot (\alpha(\mathbf{x}) \nabla), \quad (4.26)$$

(4.23) is a heat equation. Here, $\alpha_1 \geq \alpha(\mathbf{x}) \geq \alpha_0 > 0$ with some constants α_0, α_1 . The boundary condition (4.24) can be Dirichlet, Neumann, and Robin types, and the initial condition (4.25) is given by

$$u(\mathbf{x}, 0) = h_0(\mathbf{x}) \quad \text{in } \Omega. \quad (4.27)$$

Unlike the traditional approach, for example, the finite difference method, discretizing the time interval as $0 = t_0 < t_1 < \dots < t_N = T$ and sequentially solving $u(\mathbf{x}; t_n)$ for $n = 1, \dots, N$, we adopt space-time approach, i.e., temporal and spatial variables are treated jointly and equally.

4.1. RNN-PG method for the heat equation

First, we consider solving a heat equation as follows

$$\frac{\partial u}{\partial t} - \nabla \cdot (\alpha(\mathbf{x}) \nabla u) = f \quad \text{in } \Omega \times I, \quad (4.28)$$

$$u(\mathbf{x}, t) = g_D(\mathbf{x}, t) \quad \text{on } \Gamma_D \times I, \quad (4.29)$$

$$\alpha(\mathbf{x}) \nabla u(\mathbf{x}, t) \cdot \mathbf{n} = g_N(\mathbf{x}, t) \quad \text{on } \Gamma_N \times I, \quad (4.30)$$

$$u(\mathbf{x}, 0) = h_0(\mathbf{x}) \quad \text{in } \Omega. \quad (4.31)$$

The weak formulation of the problem is to find $u \in L^2(I; H_{D,0}^1(\Omega))$ with $\partial_t u \in L^2(I; L^2(\Omega))$ such that

$$a_t(u, v) = l_t(v) \quad \forall v \in L^2(I; H_{D,0}^1(\Omega)), \quad (4.32)$$

$$u(\mathbf{x}, 0) = h_0(\mathbf{x}) \quad \text{in } \Omega, \quad (4.33)$$

where

$$a_t(u, v) = \int_0^T \int_{\Omega} \left(\frac{\partial u}{\partial t} v + \alpha(\mathbf{x}) \nabla u \cdot \nabla v \right) d\mathbf{x} dt,$$

$$l_t(v) = \int_0^T \int_{\Omega} f v d\mathbf{x} dt + \int_0^T \int_{\Gamma_N} g_N v ds dt.$$

Here, $L^2(I; H_{D, g_D}^1(\Omega))$ is a Banach space of all measurable functions $v : I \rightarrow H_{D, g_D}^1$ such that

$$\left[\int_0^T \|v(t)\|_{H^1(\Omega)}^2 dt \right]^{1/2} < \infty.$$

The RNN-PG method for solving the problem (4.32)–(4.33) is to find a neural network $u_{\rho}(\mathbf{x}, t) \in U_{\rho} \subset \mathcal{N}_{\rho}(D, M_D, B_D) \subset L^2(I; H^1(\Omega))$ such that

$$a_t(u_{\rho}, v) = l_t(v) \quad \forall v \in V_h \subset L^2(I; H_{D, 0}^1(\Omega)), \quad (4.34)$$

$$u_{\rho}(\mathbf{x}_k, t_k) = g_D(\mathbf{x}_k, t_k) \quad \text{for some points } (\mathbf{x}_k, t_k) \in \Gamma_D \times I, \quad k = 1, 2, \dots, N_b, \quad (4.35)$$

$$u_{\rho}(\mathbf{x}_m, 0) = h_0(\mathbf{x}_m) \quad \text{for some points } \mathbf{x}_m \in \Omega, \quad m = 1, 2, \dots, N_c, \quad (4.36)$$

Let us construct $u_{\rho} \in U_{\rho}$ the same as the one in (2.14) except for the input dimension $n_0 = d + 1$, i.e.,

$$u_{\rho}(\mathbf{x}, t) = \sum_{j=1}^{n_{D-1}} u_{\rho}^j \Phi_j^u(\mathbf{x}, t).$$

Similarly, Eq. (4.34) becomes: Find $u_{\rho}^j, j = 1, \dots, n_{D-1}$, such that

$$\sum_{j=1}^{n_{D-1}} u_{\rho}^j a_t(\Phi_j^u(\mathbf{x}, t), v_i) = l_t(v_i) \quad \forall v_i \in V_h,$$

which gives

$$\mathbf{A}\mathbf{U} = \mathbf{L},$$

where $\mathbf{A}_{i,j} = a_t(\Phi_j^u(\mathbf{x}, t), v_i)$, $\mathbf{U} = (u_{\rho}^1, \dots, u_{\rho}^{n_{D-1}})^T$, and $\mathbf{L} = (l_1, \dots, l_{N_h})^T$ with $l_i = l_t(v_i)$ ($i = 1, \dots, N_h$). Note that $\{\Phi_j^u(\mathbf{x}, t)\}_{j=1}^{n_{D-1}}$ can be regarded as a group of bases in the space-time domain.

To deal with the boundary condition (4.35) and the initial condition (4.36), we still need to take some random samples $\{(\mathbf{x}_k, t_k)\}_{k=1}^{N_b}$ and $\{\mathbf{x}_m\}_{m=1}^{N_c}$ according to the uniform distribution $\mathcal{U}(\Gamma_D \times I)$ and $\mathcal{U}(\Omega)$, separately. Then set $u_{\rho}(\mathbf{x}_k, t_k) = g_D(\mathbf{x}_k, t_k)$ for all $(\mathbf{x}_k, t_k) \in \Gamma_D \times I$ and $u_{\rho}(\mathbf{x}_m, 0) = h_0(\mathbf{x}_m)$ for all $\mathbf{x}_m \in \Omega$, i.e.,

$$\sum_{j=1}^{n_{D-1}} u_{\rho}^j \Phi_j^u(\mathbf{x}_k, t_k) = g_D(\mathbf{x}_k, t_k) \quad \text{for } k = 1, \dots, N_b,$$

$$\sum_{j=1}^{n_{D-1}} u_{\rho}^j \Phi_j^u(\mathbf{x}_m, 0) = h_0(\mathbf{x}_m) \quad \text{for } m = 1, \dots, N_c.$$

Thus, the above equations imply $\mathbf{B}\mathbf{U} = \mathbf{G}$ and $\mathbf{C}\mathbf{U} = \mathbf{H}$, where $\mathbf{B}_{k,j} = \Phi_j^u(\mathbf{x}_k, t_k)$, $\mathbf{C}_{m,j} = \Phi_j^u(\mathbf{x}_m, 0)$, $\mathbf{G} = (g_1, \dots, g_{N_b})^T$ with $g_k = g_D(\mathbf{x}_k, t_k)$ and $\mathbf{H} = (h_1, \dots, h_{N_c})^T$ with $h_m = h_0(\mathbf{x}_m)$.

Finally, we can obtain the solution u_{ρ} by solving a least-squares problem with the linear system

$$\begin{bmatrix} \mathbf{A} \\ \mathbf{B} \\ \mathbf{C} \end{bmatrix} \mathbf{U} = \begin{bmatrix} \mathbf{L} \\ \mathbf{G} \\ \mathbf{H} \end{bmatrix}. \quad (4.37)$$

4.2. RNN-PG method for the wave equation

In this subsection, we consider the following wave equation

$$\frac{\partial^2 u}{\partial t^2} - \nabla \cdot (\alpha(\mathbf{x}) \nabla u) = f \quad \text{in } \Omega \times I, \quad (4.38)$$

$$u(\mathbf{x}, t) = g_D(\mathbf{x}, t) \quad \text{on } \Gamma_D \times I, \quad (4.39)$$

$$\alpha(\mathbf{x}) \nabla u(\mathbf{x}, t) \cdot \mathbf{n} = g_N(\mathbf{x}, t) \quad \text{on } \Gamma_N \times I, \quad (4.40)$$

$$u(\mathbf{x}, 0) = h_0(\mathbf{x}) \quad \text{in } \Omega, \quad (4.41)$$

$$\frac{\partial u(\mathbf{x}, 0)}{\partial t} = w_0(\mathbf{x}) \quad \text{in } \Omega. \quad (4.42)$$

The weak formulation of the wave equation is: Find $u \in L^2(I; H_{D,g_D}^1(\Omega))$ with $\partial_t u \in L^2(I; L^2(\Omega))$ such that

$$a_w(u, v) = l_w(v) \quad \forall v \in L^2(I; H_{D,0}^1(\Omega)), \quad (4.43)$$

$$u(\mathbf{x}, 0) = h_0(\mathbf{x}) \quad \text{in } \Omega, \quad (4.44)$$

where

$$a_w(u, v) = \int_0^T \int_{\Omega} \left(-\frac{\partial u}{\partial t} \frac{\partial v}{\partial t} + \alpha(\mathbf{x}) \nabla u \cdot \nabla v \right) d\mathbf{x} dt + \int_{\Omega} \frac{\partial u(\mathbf{x}, T)}{\partial t} v(\mathbf{x}, T) d\mathbf{x},$$

$$l_w(v) = \int_0^T \int_{\Omega} f v d\mathbf{x} dt + \int_0^T \int_{\Gamma_N} g_N v ds dt + \int_{\Omega} w_0 v(\mathbf{x}, 0) d\mathbf{x}.$$

Note that the initial condition (4.42) becomes a natural boundary condition of the domain $\Omega \times I$, and it is built into the weak formulation (4.43).

The RNN-PG method for solving the wave equation is to find a neural network $u_{\rho} \in U_{\rho} \subset \mathcal{N}_{\rho}(D, M_D, B_D) \subset L^2(I; H^1(\Omega))$ such that

$$a_w(u_{\rho}, v) = l_w(v) \quad \forall v \in V_h \subset L^2(I; H_{D,0}^1(\Omega)),$$

$$u_{\rho}(\mathbf{x}_k, t_k) = g_D(\mathbf{x}_k, t_k) \quad \text{for some points } (\mathbf{x}_k, t_k) \in \Gamma_D \times I, \quad k = 1, 2, \dots, N_b,$$

$$u_{\rho}(\mathbf{x}_m, 0) = h_0(\mathbf{x}_m) \quad \text{for some points } \mathbf{x}_m \in \Omega, \quad m = 1, 2, \dots, N_c,$$

By constructing $u_{\rho}(\mathbf{x}, t)$ and making restrictions on the boundary and initial conditions in a similar way for the heat equation, we obtain a similar linear system as (4.37), then solve the least-squares problem to get solution u_{ρ} .

Remark 4.1.

In general, it is often preferable to decompose a large problem into smaller ones and solve them individually to reduce memory usage and overall computation time. For time-dependent problems, finite difference discretization for the temporal variable is a popular choice as it follows this divide-and-conquer strategy. However, a drawback of this approach is the accumulation of errors during the step-by-step process. To avoid this issue, the space-time finite element method (FEM) can be used, but it requires significant computational resources. With the strong approximation ability of neural networks, the RNN-PG method has the potential to solve time-dependent problems efficiently and accurately using the space-time approach.

4.3. RNN-PG method for Burger's equation

In previous sections, we demonstrated the use of RNN-PG methods to solve linear partial differential equations. In this subsection, we illustrate how to apply the RNN-PG method to solve nonlinear PDEs using Burger's equation as an example. The 2-D Burger's equation is given as follows:

$$\frac{\partial u}{\partial t} + u \left(\frac{\partial u}{\partial x} + \frac{\partial u}{\partial y} \right) - \Delta u = f \quad \text{in } \Omega \times I, \quad (4.45)$$

$$u(\mathbf{x}, t) = g_D(\mathbf{x}, t) \quad \text{on } \Gamma_D \times I, \quad (4.46)$$

$$\alpha(\mathbf{x}) \nabla u(\mathbf{x}, t) \cdot \mathbf{n} = g_N(\mathbf{x}, t) \quad \text{on } \Gamma_N \times I, \quad (4.47)$$

$$u(\mathbf{x}, 0) = h_0(\mathbf{x}) \quad \text{in } \Omega. \quad (4.48)$$

To address the above problem, we can employ nonlinear iteration methods such as Picard iteration and Newton's method. The weak formulation of the problem with Picard iteration is: given $u^0 \in L^2(I; H_{D,g_D}^1(\Omega))$, we seek $u^{n+1} \in L^2(I; H_{D,g_D}^1(\Omega))$ with $\partial_t u^{n+1} \in L^2(I; L^2(\Omega))$ such that

$$a_p(u^n, u^{n+1}, v) = l_p(v) \quad \forall v \in L^2(I; H_{D,0}^1(\Omega)), \quad (4.49)$$

$$u^{n+1}(\mathbf{x}, y, 0) = h_0(\mathbf{x}, y, t) \quad \text{in } \Omega, \quad (4.50)$$

with $n = 0, 1, 2, \dots$, where

$$a_p(u^n, u^{n+1}, v) = \int_0^T \int_{\Omega} \left(\frac{\partial u^{n+1}}{\partial t} v + u^n \left(\frac{\partial u^{n+1}}{\partial x} + \frac{\partial u^{n+1}}{\partial y} \right) v + \nabla u^{n+1} \cdot \nabla v \right) d\mathbf{x} dt,$$

$$l_p(v) = \int_0^T \int_{\Omega} f v d\mathbf{x} dt + \int_0^T \int_{\Gamma_N} g_N v ds dt.$$

The weak formulation of the problem with the Netwon iteration is: Given $u^0 \in L^2(I; H_{D,0}^1(\Omega))$, we find $u^{n+1} \in L^2(I; H_{D,0}^1(\Omega))$ with $\partial_t u^{n+1} \in L^2(I; L^2(\Omega))$ such that

$$a_e(u^n, u^{n+1}, v) = l_e(u^n, v) \quad \forall v \in L^2(I; H_{D,0}^1(\Omega)), \quad (4.51)$$

$$u^{n+1}(x, y, 0) = h_0(x, y, t) \quad \text{in } \Omega, \quad (4.52)$$

with $n = 0, 1, 2, \dots$, where

$$\begin{aligned} a_e(u^n, u^{n+1}, v) &= \int_0^T \int_{\Omega} \left(\frac{\partial u^{n+1}}{\partial t} v + \left(\frac{\partial u^n}{\partial x} + \frac{\partial u^n}{\partial y} \right) u^{n+1} v + u^n \left(\frac{\partial u^{n+1}}{\partial x} + \frac{\partial u^{n+1}}{\partial y} \right) v + \nabla u^{n+1} \cdot \nabla v \right) d\mathbf{x} dt, \\ l_e(u^n, v) &= \int_0^T \int_{\Omega} f v + u^n \left(\frac{\partial u^n}{\partial x} + \frac{\partial u^n}{\partial y} \right) v d\mathbf{x} dt + \int_0^T \int_{\Gamma_N} g_N v ds dt. \end{aligned}$$

Then we can use the RNN-PG method to solve the above problem similarly as Eq. (4.34)–(4.36).

5. Numerical examples

In this section, we provide several examples to demonstrate the performance of RNN-PG methods. In these examples, we consider $d = 1$ or 2 and use the basis functions of the finite element method as test functions. Specifically, we decompose the domain Ω and $\Omega \times I$ into square or cubic elements and adopt corresponding bilinear or trilinear functions as test functions. Numerical integration is calculated by Gauss–Legendre quadrature with 25 points in each square and 125 points in each cubic. We also consider other types of test functions to compare their performance with finite element basis in Example 5.1.

To calculate ∇u_ρ , one can use automatic differentiation which can handle derivatives for a wide range of functions. However, for faster computation, we use the difference method, specifically the central difference for a first-order derivative given by:

$$\frac{\partial u_\rho(x, y)}{\partial x} = \frac{u_\rho(x + h_x, y) - u_\rho(x - h_x, y)}{2h_x}.$$

In the following numerical experiments, we choose $h_x = h_y = 10^{-6}$.

The initialization of weight parameters is crucial for RNN-PG methods. Once initialized, these weights remain fixed except for the last layer. To ensure reasonable initialization, we can use the Pytorch function *torch.nn.init*. In our examples, we use the uniform distribution, kaiming uniform distribution [49] and Xavier uniform distribution [50] for a two-layer fully connected network, three-layer fully connected network, and ResNet, respectively. We use the activation function $\rho = \tanh$ in both two-layer fully connected network and ResNet to achieve a smooth approximation of the PDE solution.

Example 5.1. In this example, we solve a 2-dimensional Poisson equation with a smooth solution $u = \cos(\pi x) \sin(\pi y)$ under the mixed boundary conditions

$$\begin{cases} -\Delta u(x, y) = 2\pi^2 \cos(\pi x) \sin(\pi y) & \text{in } \Omega, \\ u(x, y) = 0 & \text{on } \Gamma_D, \\ \nabla u(x, y) \cdot \mathbf{n} = 0 & \text{on } \Gamma_N, \end{cases}$$

where $\Omega = [0, 1]^2$, $\Gamma_D = (0, 1) \times \{0, 1\}$, $\Gamma_N = \{0, 1\} \times (0, 1)$.

For the RNN-PG method, we use a two-layer fully connected neural network with uniform distribution $\mathcal{U}(-1, 1)$ for initialization, $D = 2$, $n_0 = 2$, $n_2 = 1$, and we choose $n_1 = 50, 100, 200$. The degrees of freedom (DoF) for the RNN-PG method is the number of neurons in the first layer, that is, n_1 . We choose test functions as bases of bilinear finite elements on a family of square meshes with mesh size $h = 2^{-n}$ ($n = 2, 3, 4, 5$). We calculate the numerical integration by Gauss–Legendre quadrature with 25 points inside each square and randomly sample 100 points on each edge of Γ_D . For solving the resulting linear system, we adopt the least-squares solver *scipy.linalg.lstsq* in Python.

To demonstrate the accuracy of the RNN-PG method, we show the numerical solution u_ρ , exact solution u , and their difference $|u - u_\rho|$ in Fig. 2. Fig. 2(a) displays the numerical solution u_ρ solved by the RNN-PG method with $h = 2^{-5}$ and DoF = 200, and its relative L^2 error and H^1 error are 9.651×10^{-10} and 5.167×10^{-9} , respectively. The exact solution u is given in Fig. 2(b), and Fig. 2(c) shows the absolute value of their difference $|u - u_\rho|$, indicating that the maximum error is around 10^{-8} . We observe that the RNN-PG method provides highly accurate numerical solutions.

In Table 1, we report the relative L^2 and H^1 errors, denoted by e_{l2} and e_{H^1} , respectively, for $h = 2^{-n}$ ($n = 2, 3, 4, 5$) and DoF = 50, 100, 200. RNN-PG method provides more accurate numerical solutions with a larger number of DoF = n_1 , given the same data of test functions. Moreover, fixing the number of DoF and decreasing the mesh size h leads to a reduction in errors, i.e., more test functions are fed into the system, and the neural networks supply more accurate numerical solutions.

To compare the performance of the RNN-PG method with the finite element method, we compute numerical solutions using FEM through Fenics [51] on a family of uniform triangulations with mesh size $h = 2^{-n}$ ($n = 2, 3, 4, 5$). We use

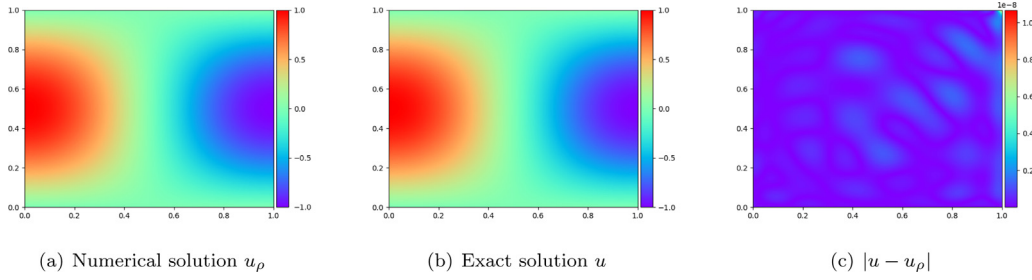


Fig. 2. Numerical solution u_ρ by RNN-PG method with $h = 2^{-5}$ and DoF = 200 in Example 5.1.

Table 1

Relative $L^2(\Omega)$ and $H^1(\Omega)$ errors of the RNN-PG method with different h and DoF in Example 5.1.

h	DoF					
	50		100		200	
	e_{L^2}	e_{H^1}	e_{L^2}	e_{H^1}	e_{L^2}	e_{H^1}
2^{-2}	6.895e-3	2.080e-2	4.745e-3	1.507e-2	2.931e-3	9.062e-3
2^{-3}	4.604e-5	3.161e-4	1.116e-5	7.537e-5	4.081e-6	2.732e-5
2^{-4}	2.892e-5	1.139e-4	2.684e-8	1.219e-7	2.890e-9	1.383e-8
2^{-5}	2.730e-5	9.634e-5	1.219e-8	5.417e-8	9.651e-10	5.167e-9

Table 2

Relative $L^2(\Omega)$ and $H^1(\Omega)$ errors of the P_k FEM with different h in Example 5.1.

h	Scheme								
	P_1 Linear FEM			P_2 quadratic FEM			P_3 cubic FEM		
	DoF	e_{L^2}	e_{H^1}	DoF	e_{L^2}	e_{H^1}	DoF	e_{L^2}	e_{H^1}
2^{-2}	15	1.829e-1	3.990e-1	63	8.878e-3	5.615e-2	143	6.626e-4	5.724e-3
2^{-3}	63	4.399e-2	1.934e-1	255	1.102e-3	1.456e-2	575	3.958e-5	7.202e-4
2^{-4}	255	1.091e-2	9.601e-2	1023	1.375e-4	3.683e-3	2303	2.417e-6	9.004e-5
2^{-5}	1023	2.721e-3	4.792e-2	4095	1.718e-5	9.246e-4	9215	1.495e-7	1.125e-5

Table 3

Relative $L^2(\Omega)$ and $H^1(\Omega)$ errors of the RNN-PG method with respect to DoF and the number of test functions (NTF) for a fixed mesh size $h = 2^{-5}$ in Example 5.1.

NTF	DoF					
	50		100		200	
	e_{L^2}	e_{H^1}	e_{L^2}	e_{H^1}	e_{L^2}	e_{H^1}
200	1.274e-4	1.861e-4	3.043e-7	8.408e-7	1.364e-7	1.570e-7
400	1.244e-4	2.093e-4	1.449e-7	3.028e-7	1.435e-8	2.177e-8
600	1.005e-4	2.881e-4	8.909e-8	1.573e-7	3.745e-9	1.387e-8
800	7.368e-5	2.020e-4	4.091e-8	1.751e-7	3.968e-9	1.065e-8
1023	3.914e-5	1.437e-4	2.597e-8	1.039e-7	9.651e-10	5.167e-9

the standard triangle Lagrange elements P_k ($k = 1, 2, 3$) for solving the problem, where k denotes the degree of the polynomials. The degrees of freedom for P_k FEM in this example is $\text{DoF} = (k/h)^2 - 1$. We list the numerical errors in Table 2. Comparing the accuracy concerning degrees of freedom across Tables 1 and 2, we observe that the RNN-PG method outperforms FEM, yielding more accurate numerical solutions with significantly fewer degrees of freedom.

Next, we explore the impact of network depth and data on the results of the RNN-PG method. Data is determined by the test functions, and changing the mesh size or choosing different numbers of test functions generates different data, while the network can be formed by different depths and widths. We fix the mesh size to $h = 2^{-5}$ and chose test functions randomly without replacement, then we report relative errors $L^2(\Omega)$ and $H^1(\Omega)$ for different numbers of unfixed neurons (DoF) and numbers of test functions (NTF) in Table 3, demonstrating that more data information results in more precise solutions. We then initialize ResNet with Xavier uniform distribution $\tilde{U}(-1, 1)$ and change the number of layers D from 2 to 5, reporting the relative errors $L^2(\Omega)$ and $H^1(\Omega)$ for different neurons (DoF) in each layer in Table 4, and find that deeper neural networks produce better numerical solutions.

Table 5 presents a comparison of RNN-PG method results using different types of test functions, namely piecewise bilinear functions, randomized neural networks, Legendre polynomials, and Chebyshev polynomials. We use 100 test functions of each type with a fixed degree of freedom of 100. The results demonstrate that using piecewise bilinear

Table 4

Relative $L^2(\Omega)$ and $H^1(\Omega)$ errors of the RNN-PG method with respect to DoF and the depth D for a fixed mesh size $h = 2^{-5}$ in Example 5.1.

D	DoF					
	50		100		200	
	e_{L^2}	e_{H^1}	e_{L^2}	e_{H^1}	e_{L^2}	e_{H^1}
2	1.178e-5	4.100e-5	5.827e-6	1.405e-5	9.318e-5	2.054e-4
3	1.093e-5	3.165e-5	1.661e-6	4.765e-6	1.342e-6	3.937e-6
4	1.302e-5	4.786e-5	1.857e-7	7.485e-7	1.240e-7	4.744e-7
5	6.548e-5	2.674e-4	4.485e-8	2.093e-7	4.629e-8	2.108e-7

Table 5

Some properties of the RNN-PG method with respect to different types of test functions in Example 5.1.

Test functions v	DoF	Rank of linear system	Condition number	Relative errors
FE bilinear functions	100	93	2.6e12	2.859e-7
Neural networks	100	92	9.1e16	3.437e-5
Legendre polynomials	100	63	3.2e19	8.002e-2
Chebyshev polynomials	100	61	1.8e19	9.602e-2

Table 6

Relative $L^2(\Omega)$ and $H^1(\Omega)$ errors of four Mixed RNN-PG methods with different h and DoF in Example 5.1.

h	Scheme								
h	DoF	M-RNN-PG method 1		M-RNN-PG method 2		M-RNN-PG method 3		M-RNN-PG method 4	
		e_{L^2}	e_{H^1}	e_{L^2}	e_{H^1}	e_{L^2}	e_{H^1}	e_{L^2}	e_{H^1}
2^{-2}	75	4.408e-2	1.315e-1	7.098e-1	1.677e-1	3.785e-3	1.754e-2	7.333e-2	7.569e-2
2^{-3}	150	4.857e-5	2.063e-4	8.316e-5	2.287e-4	3.103e-5	9.329e-5	5.396e-5	1.268e-4
2^{-4}	300	9.944e-8	2.984e-7	5.140e-8	4.180e-7	4.989e-8	3.502e-7	1.995e-8	1.004e-7
2^{-5}	600	3.503e-9	7.141e-8	2.783e-9	3.795e-8	4.086e-9	4.536e-8	1.395e-10	1.678e-9

functions results in a smaller condition number for the linear system. In contrast, the condition number of the linear system is larger for the other three cases. Hence, constructing suitable preconditioners to enhance the accuracy of DGPM is another intriguing topic.

In Table 6, we compare the performance of mixed RNN-PG methods with different formulations. All four schemes show improved results with higher degrees of freedom. M-RNN-PG method 4 achieves a relative L^2 error as low as 1.395×10^{-10} , demonstrating its high accuracy. It is noteworthy that M-RNN-PG method 4 does not require the calculation of derivatives for u and $\mathbf{p}$, and boundary conditions are naturally incorporated into the formulation, eliminating the need for boundary restrictions.

Example 5.2. Given $\Omega = (0, 1)^2$ with $\Gamma_D = \partial\Omega := \Gamma_1 \cup \Gamma_2 \cup \Gamma_3 \cup \Gamma_4$, we consider the following heat equation

$$\begin{cases} u_t(x, y, t) - \Delta u(x, y, t) = f(x, y, t) & \text{in } \Omega \times I, \\ u(x, y, t) = g(x, y, t) & \text{on } \Gamma_D \times I, \\ u(x, y, 0) = h_0(x, y) & \text{in } \Omega, \end{cases}$$

with $I = (0, 1)$. The exact solution $u(x, y, t) = 2e^{-t} \sin(\frac{\pi}{2}x) \sin(\frac{\pi}{2}y)$ and the right-hand term $f(x, y, t)$ is given accordingly.

As we mentioned in Section 4, instead of discretizing the temporal variable by finite difference method, the RNN-PG method solves time-dependent equations directly in the space-time domain. In this example, the RNN-PG method treats the above heat equation as a 3-dimensional problem with a 2-dimensional spatial variable and a 1-dimensional temporal variable. In the test, we initialize a two-layer fully connected neural network with uniform distribution $\mathcal{U}(-1, 1)$ and set $D = 2$, $n_0 = 3$, and $n_2 = 1$. We choose $n_1 = 200, 400, 800$ and use trilinear functions on cubic meshes as test functions. We calculate numerical integration using the Gauss-Legendre quadrature with 125 points inside each cubic. To enforce Dirichlet and initial conditions, we randomly sample 100 points on each of the five faces of the space-time domain, namely $\Gamma_1 \times I$, $\Gamma_2 \times I$, $\Gamma_3 \times I$, $\Gamma_4 \times I$, and $\Omega \times 0$. Table 7 shows L^2 and H^1 errors at the ending time $t = 1$. The results demonstrate that the RNN-PG method is effective in solving the heat equation with high accuracy.

To investigate whether using Gauss-Legendre points as collocation points improves the results, we fix the mesh size to $h = 2^{-4}$ and vary the number of Gauss-Legendre points on each face of the space-time domain to enforce the boundary and initial conditions. We report the errors in L^2 and H^1 norms at the final time $t = 1$ in Table 8. We can see that increasing the number of collocation points leads to more accurate results. Comparing Tables 7 and 8, we observe that using Gauss-Legendre points as collocation points gives slightly better accuracy, but with the same order of magnitude.

Table 7
 $L^2(\Omega)$ and $H^1(\Omega)$ errors of the RNN-PG method with different h and DoF in Example 5.2.

h	DoF					
	200		400		800	
	L^2 error	H^1 error	L^2 error	H^1 error	L^2 error	H^1 error
2^{-2}	5.537e-5	7.786e-4	5.346e-5	7.592e-4	2.985e-5	4.259e-4
2^{-3}	3.792e-5	3.261e-4	1.882e-7	2.743e-6	1.652e-8	5.266e-7
2^{-4}	2.110e-5	1.633e-4	1.674e-7	2.917e-6	9.499e-10	1.635e-8

Table 8
 $L^2(\Omega)$ and $H^1(\Omega)$ errors of the RNN-PG method with respect to DoF and the number of Gauss-Legendre collocation points (NCP) on each face of the space-time domain with mesh size $h = 2^{-4}$ in Example 5.2.

NCP	DoF					
	200		400		800	
	L^2 error	H^1 error	L^2 error	H^1 error	L^2 error	H^1 error
25	7.375e-5	4.836e-4	1.176e-5	8.893e-5	3.986e-6	2.699e-5
49	3.607e-5	2.930e-4	9.603e-7	7.728e-6	6.989e-8	6.594e-7
100	2.527e-5	2.284e-4	9.436e-8	9.607e-7	9.043e-10	1.144e-8

Table 9
 $L^2(\Omega)$ and $H^1(\Omega)$ errors of the P_2 FEM for spatial discretization with different h and back Euler scheme for temporal discretization with different time-step Δt in Example 5.2.

h	Δt					
	10^{-3}		2×10^{-4}		5×10^{-5}	
	L^2 error	H^1 error	L^2 error	H^1 error	L^2 error	H^1 error
2^{-5}	7.268e-6	3.481e-5	1.450e-6	7.209e-6	3.597e-7	2.657e-6
2^{-6}	7.272e-6	3.480e-5	1.454e-6	6.960e-6	3.632e-7	1.756e-6
2^{-7}	7.272e-6	3.480e-5	1.454e-6	6.958e-6	3.635e-7	1.739e-6

We compare the performance of the RNN-PG method with traditional numerical methods by considering a numerical scheme that uses P_2 finite element discretization for the spatial variable and back Euler finite difference approximation for the temporal variable. We choose time-steps of $\Delta t = 10^{-3}, 2 \times 10^{-4}, 5 \times 10^{-5}$ and mesh-sizes of $h = 2^{-5}, 2^{-6}, 2^{-7}$. Then, we report the L^2 and H^1 errors at the ending time $t = 1$, see Table 9. RNN-PG method achieves more accurate numerical solutions compared to FEM, and is computationally less expensive than traditional approaches.

Example 5.3. Consider the following wave equation

$$\begin{cases} \frac{\partial^2 u}{\partial t^2}(x, y, t) - \Delta u(x, y, t) = f(x, y, t) & \text{in } \Omega \times (0, 1), \\ u(x, y, t) = g(x, y, t) & \text{on } \Gamma_D \times (0, 1), \\ u(x, y, 0) = h_0(x, y) & \text{in } \Omega, \\ \frac{\partial u}{\partial t}(x, y, 0) = w_0(x, y) & \text{in } \Omega, \end{cases}$$

with the exact solution $u(x, y, t) = \sin(\frac{\pi}{2}x)\sin(\frac{\pi}{2}y)\sin(\frac{\pi}{2}t)$ over $\Omega = (0, 1)^2 \subset \mathbb{R}^2$ and $\Gamma_D = \partial\Omega$. The right-hand term $f(x, y, t)$ is given accordingly.

In this example, we use the RNN-PG method based on the space-time approach to solve the wave equation. We initialize a two-layer fully connected neural network with a uniform distribution $\mathcal{U}(-1, 1)$ and choose $D = 2, n_0 = 3$, and $n_2 = 1$. We also choose $n_1 = 200, 400, 800$ separately, and the test functions are trilinear functions on cubic meshes. We randomly sample 100 points on each face to enforce the Dirichlet boundary and initial conditions and calculate numerical integration using Gauss-Legendre quadrature with 125 points inside each cubic. The L^2 and H^1 errors at the ending time $t = 1$ are shown in Table 10, indicating that the RNN-PG method obtains very accurate numerical solutions.

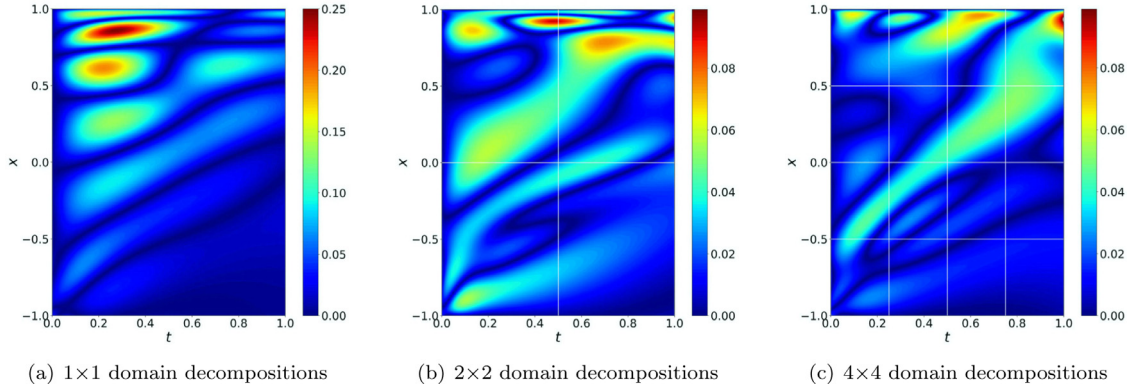
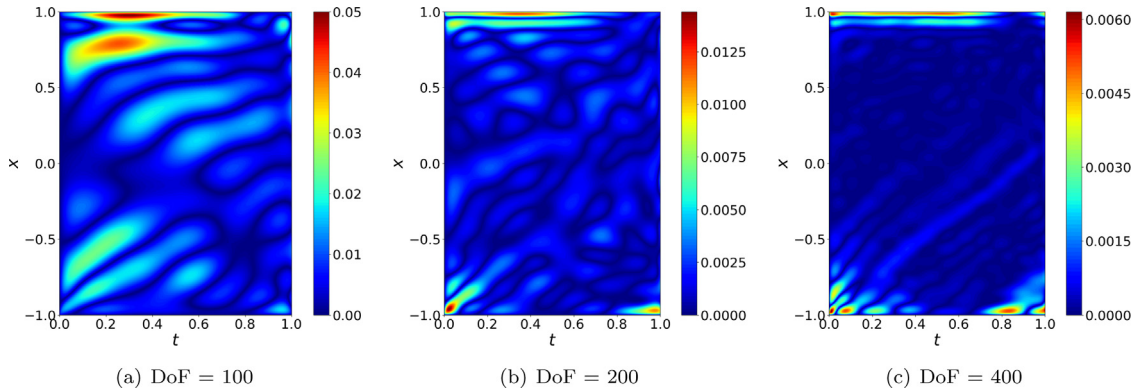
Example 5.4. Given $\Omega = (-1, 1)$, we consider a one-dimensional advection-diffusion equation

$$\begin{cases} \frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} = \kappa \frac{\partial^2 u}{\partial x^2} & \text{in } \Omega \times I, \\ u(-1, t) = u(1, t) = 0 & \text{in } I, \\ u(x, 0) = -\sin(\pi x) & \text{in } \Omega, \end{cases}$$

with $I = (0, 1)$, and $\kappa = 0.1/\pi$.

Table 10
 $L^2(\Omega)$ and $H^1(\Omega)$ errors of the RNN-PG method with different h and DoF in Example 5.3.

h	DoF					
	200		400		800	
	L^2 error	H^1 error	L^2 error	H^1 error	L^2 error	H^1 error
2^{-2}	7.422e-5	1.046e-3	7.435e-5	1.059e-3	7.478e-5	1.053e-3
2^{-3}	1.800e-5	1.767e-4	1.052e-7	2.563e-6	1.904e-8	5.813e-7
2^{-4}	1.091e-5	1.355e-4	1.068e-7	1.346e-6	1.395e-9	4.394e-8

**Fig. 3.** Pointwise error of hp -VPINN with different domain decompositions in Example 5.4.
Source: Taken from [25].**Fig. 4.** Pointwise error for RNN-PG method with $h = 2^{-5}$ and different degrees of freedoms in Example 5.4.

We consider this equation as a 2-dimensional problem and solve it directly in the space-time domain using the RNN-PG method. In the test, we initialize a three-layer fully connected neural network with kaiming uniform distribution and use Relu⁵ and tanh as activation functions for the 1st and 2nd layer, respectively. We set $D = 3$, $n_0 = 2$, $n_3 = 1$, and $n_1 = n_2 = 100, 200, 400$. The test functions are bilinear functions on square meshes with mesh size $h = 2^{-5}$ and we calculate numerical integration by Gauss-Legendre quadrature with 25 points inside each square. We also randomly sample 100 points on I and Ω to enforce the boundary and initial conditions. The reference solution is obtained through a series summation with 800 terms of the exact solution in [52].

Fig. 3 shows the results of the hp -VPINN method with different domain decompositions in [25]. They use individual networks in each sub-domain with four layers and each with 5 neurons, resulting in 1296 training parameters for the case in Fig. 3(c). In contrast, Fig. 4 shows the results of the RNN-PG method, which computes the problem in the whole domain with a least-squares method, achieving faster and more accurate results. Moreover, the RNN-PG method can satisfy the boundary condition and initial condition without penalty parameters. The results demonstrate that the RNN-PG method achieves comparable results to the hp -VPINN method with a DoF of 100 and a more accurate numerical solution with a DoF of 400.

Table 11Errors of the RNN-PG method with different h and DoF when $\varepsilon = 1$ in Example 5.5.

h	DoF						
h	τ	200		400		800	
		L^2 error	H^1 error	L^2 error	H^1 error	L^2 error	H^1 error
2^{-2}	10^{-6}	1.513e-7	2.169e-6	1.479e-7	2.199e-6	1.006e-7	1.464e-6
2^{-3}	10^{-7}	1.415e-8	1.037e-7	1.335e-8	1.012e-7	1.292e-8	1.321e-7
2^{-4}	10^{-8}	8.232e-9	5.875e-8	8.043e-9	5.753e-8	7.312e-9	5.229e-8

Example 5.5. Consider a 2D Burger's equation as follows,

$$\begin{cases} u_t + u(u_x + u_y) - \varepsilon \Delta u = 0 & \text{in } \Omega \times (0, 1), \\ u(x, y, t) = g(x, y, t) & \text{on } \Gamma_D \times (0, 1), \\ u(x, y, 0) = h_0(x, y) & \text{in } \Omega, \end{cases} \quad (5.53)$$

with the exact solution $u(x, y, t) = 1/(1 + e^{\frac{x+y-t}{2\varepsilon}})$ over $\Omega = (0, 1)^2 \subset \mathbb{R}^2$ and $\Gamma_D = \partial\Omega$, and $\varepsilon = 1, 0.1$.

To solve Burger's equation using the space-time RNN-PG method, a two-layer fully connected neural network is initialized with a uniform distribution $\mathcal{U}(-1, 1)$. The neural network has 3 input neurons ($n_0 = 3$) and 1 output neuron ($n_2 = 1$). Trilinear functions on cubic meshes with different h are chosen as the test functions, and 100 points are randomly sampled on each face to enforce the Dirichlet boundary and initial conditions. Numerical integration is calculated by Gauss-Legendre quadrature with 125 points inside each cubic. We solve the nonlinear problem firstly using a five-step Picard iteration with zero initialization, and then followed by a Newton iteration with the result of the Picard iteration as initialization. The termination condition is $\|u^{n+1} - u^n\| \leq \tau$. When $\varepsilon = 1$, Table 11 shows the L^2 and H^1 errors at the ending time $t = 1$ for different terminal conditions, h , and degrees of freedom. The RNN-PG method achieves high accuracy in solving this nonlinear problem. In comparison, PINNs with a four-layer fully connected network with 20 hidden neurons in each layer, are trained using the Adam method for 10000 iterations, yielding an L^2 error of 3.519×10^{-3} at $t = 1$.

When $\varepsilon = 1$, $h = 2^{-3}$, DoF = 400, $\tau = 10^{-7}$, RNN-PG method obtains the numerical solution u_ρ at time $t = 1$ after three Newton iterations, as shown in Fig. 5(a), with L^2 error and H^1 error of 1.335×10^{-8} and 1.012×10^{-7} , respectively. The exact solution u at time $t = 1$ is presented in Fig. 5(b), and the absolute difference $|u - u_\rho|$ is displayed in Fig. 5(c), with a maximum error of around 10^{-8} . In Fig. 5(d), we display the progressive decrease of the L^2 error through iterations at time $t = 1$ with different numbers of neurons, including the five-step Picard iteration with a termination condition of $\tau = 10^{-7}$. It can be observed that the L^2 error decreases rapidly after the sixth step.

We consider the case of $\varepsilon = 0.1$, $h = 2^{-3}$, DoF = 400, $\tau = 0.1$. Fig. 6 displays the numerical solution, the exact solution, and the absolute difference at time $t = 1$ in subfigures (a), (b), and (c), respectively. The subfigure (d) shows the progressive decrease of the L^2 error through iterations at time $t = 1$, which reaches 5.193×10^{-3} after two Newton iterations.

Example 5.6. We follow one example in [53] to study a high-dimensional heat equation with zero source term and exact solution $u = \frac{\|\mathbf{x}\|^2}{d} + 2t$.

$$\begin{cases} u_t(\mathbf{x}, t) - \Delta u(\mathbf{x}, t) = f & \text{in } \Omega \times I, \\ u(\mathbf{x}, t) = g(\mathbf{x}, t) & \text{on } \Gamma_D \times I, \\ u(\mathbf{x}, 0) = h_0(\mathbf{x}) & \text{in } \Omega, \end{cases}$$

where $\Omega = [0, 1]^d$ with $\Gamma_D = \partial\Omega$, $I = [0, 1]$.

We initialize a two-layer fully connected neural network with uniform distribution $\mathcal{U}(-0.01, 0.01)$ and set $D = 2$, $n_0 = d + 1$, and $n_2 = 1$. The hidden layer has $n_1 = 800, 1600, 3200$ neurons. We choose 8192 Dirac delta functions as test functions by the Monte Carlo method, such that $v(\mathbf{x}, t) = \delta(\mathbf{x} - \mathbf{x}_0)\delta(t - t_0)$, $(\mathbf{x}_0, t_0) \in \Omega \times I$. Then we sample 2048 points on $\Gamma_D \times I$ and 6144 points on Ω randomly to enforce the boundary and initial conditions. We run this experiment on CPU and show the total runtimes and L_2 errors (computed on a test set of 10^5 random samples) in Table 12. We observe that the RNN-PG method outperforms PINNs [53] in efficiency and accuracy.

6. Summary and discussion

In this paper, we propose the Randomized Neural Network with Petrov-Galerkin Methods for solving linear and nonlinear partial differential equations. Based on variational formulations, RNN-PG methods approximate the trial function space by neural networks, while the test function space can be provided flexibly by other numerical methods or neural networks. Moreover, we avoid the discrete inf-sup condition issue in the PG method by using a least-squares approach. RNN-PG methods offer several advantages, including: (i) RNN-PG methods provide much more accurate numerical

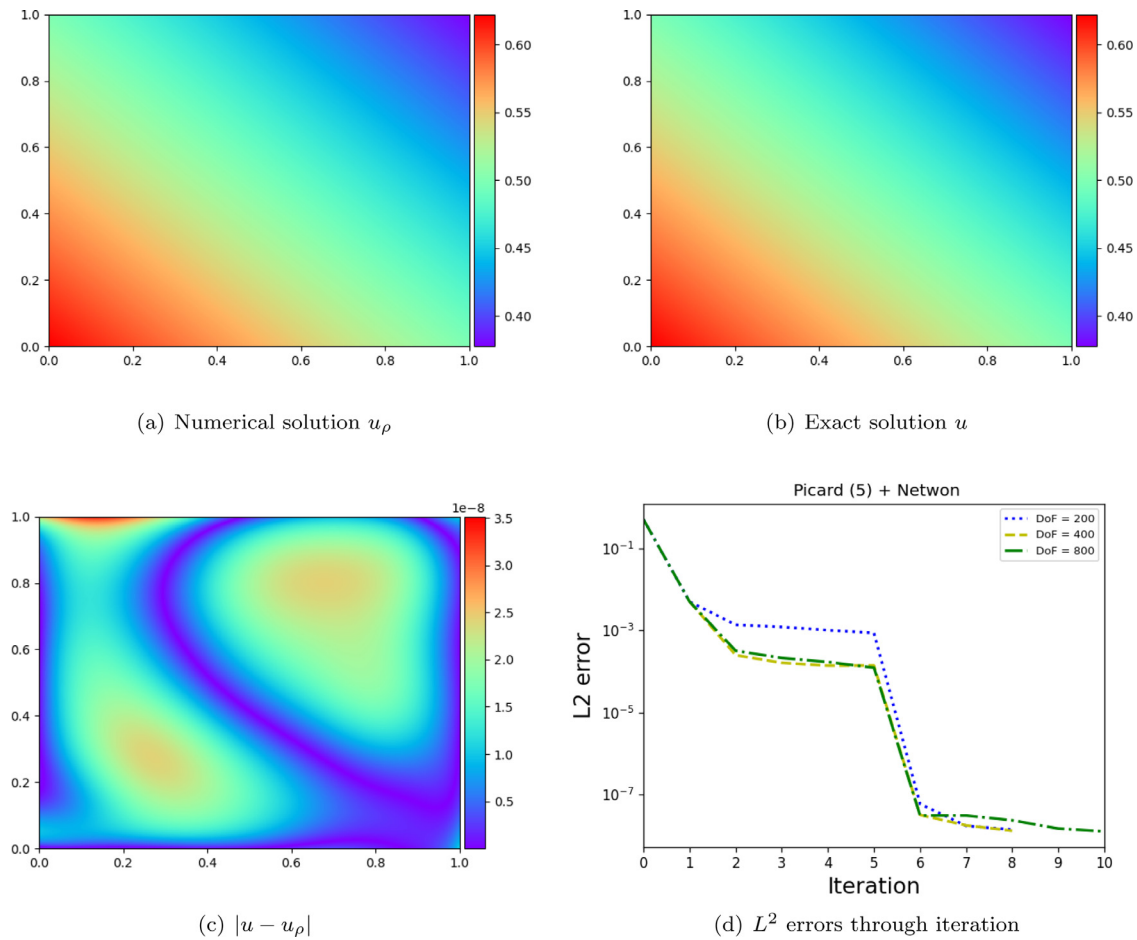


Fig. 5. When $\varepsilon = 1$, numerical solution u_ρ by RNN-PG method with $h = 2^{-3}$ and DoF = 400 in Example 5.5.

Table 12

$L^2(\Omega)$ errors and running time of the RNN-PG method with different d and DoF in Example 5.6.

d	DoF					
	800		1600		3200	
	L^2 error	Time (s)	L^2 error	Time (s)	L^2 error	Time (s)
5	2.687e-6	1.50	1.539e-6	3.53	8.529e-7	10.28
10	1.023e-5	1.92	4.411e-6	4.32	2.710e-6	12.42
20	1.352e-2	2.64	5.673e-3	5.86	5.396e-5	15.50

solutions in terms of degrees of freedom due to the powerful approximation property of neural networks; (ii) it is mesh-free, and the choice of test functions is flexible; (iii) boundary conditions can be treated easily in this framework; (iv) mixed RNN-PG methods can be constructed easily to approximate several unknown functions simultaneously, with good flexibility to handle different boundary conditions; (v) RNN-PG methods can naturally and efficiently solve linear and nonlinear time-dependent problems by the space-time approach; (vi) RNN-PG methods can solve high-dimensional problems very fast with nice accuracy.

We are optimistic about the potential of this new method for solving various partial differential equations, but several aspects require further investigation. For example, how can we design a randomized neural network to reduce the distance between the exact solution u and the trial function space U_ρ ? Specifically, when the neural network function space U_ρ fails to capture the features of the exact solution, we may need to design networks with multi-layer and multi-scale structures to improve the representation of the trial function space. Alternatively, inspired by the idea of local ELM, we can decompose the domain into several subdomains and use different neural networks to approximate different parts of the solution. More investigation is needed to overcome this difficulty in the future. The resulting linear system may have a large condition number, so how can we design neural networks and select suitable test functions to avoid this

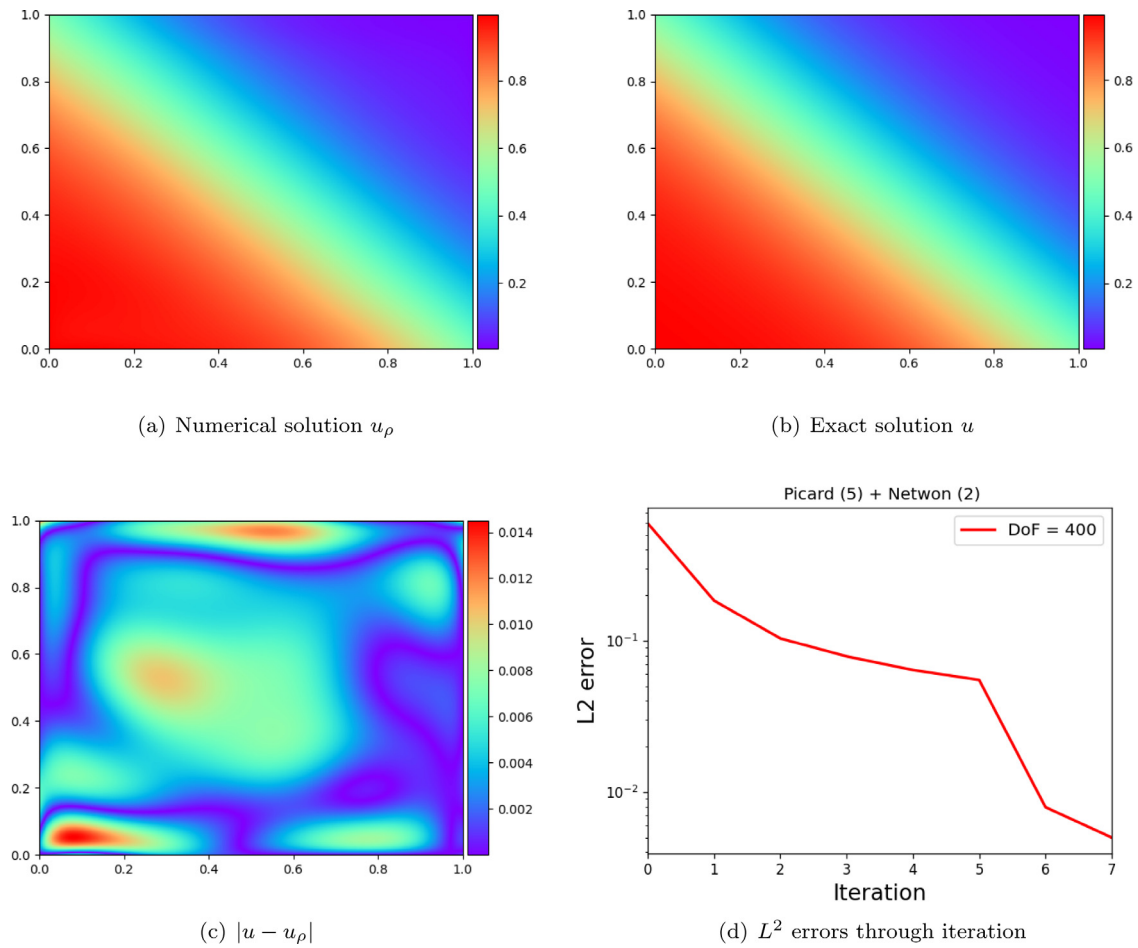


Fig. 6. When $\varepsilon = 0.1$, numerical solution u_ρ by RNN-PG method with $h = 2^{-3}$ and DoF = 400 in Example 5.5.

situation? How can we develop adaptive RNN-PG methods to improve its efficiency? How does this method perform when solving other more complex problems? Can we parallelize its implementation for solving large-scale problems? Further exploration is necessary to fully understand the numerical analysis of this method.

CRediT authorship contribution statement

Yong Shang: Methodology, Software, Data curation, Writing – original draft, Visualization. **Fei Wang:** Conceptualization, Methodology, Investigation, Writing – original draft, Writing – review & editing, Project administration, Funding acquisition, Supervision. **Jingbo Sun:** Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

We thank the three anonymous referees for their valuable comments and suggestions.

References

- [1] E W, Han J, Jentzen A. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Commun Math Stat* 2017;5:349–80.
- [2] Berg J, Nyström K. A unified deep artificial neural network approach to partial differential equations in complex geometries. *Neurocomputing* 2018;317:28–41.
- [3] Han J, Jentzen A, E W. Solving high-dimensional partial differential equations using deep learning. *Proc Natl Acad Sci* 2018;115:8505–10.
- [4] He J, Li L, Xu J, Zheng C. Relu deep neural networks and linear finite elements. *J Comput Math* 2020;38:502–27.
- [5] E W, Yu B. The Deep Ritz method: a deep learning-based numerical algorithm for solving variational problems. *Commun Math Stat* 2018;6:1–12.
- [6] Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J Comput Phys* 2019;378:686–707.
- [7] Chen J, Du R, Li P, Lyu L. Quasi-Monte Carlo sampling for machine-learning partial differential equations. 2019, arXiv Preprint arXiv:1911.01612.
- [8] Fan Y, Bohorquez CO, Ying L. BCR-Net: a neural network based on the nonstandard wavelet form. *J Comput Phys* 2019;384:1–15.
- [9] Khoo Y, Lu J, Ying L. Solving for high-dimensional committor functions using artificial neural networks. *Res Math Sci* 2019;6:1–13.
- [10] Long Z, Lu Y, Dong B. PDE-Net 2.0: learning PDEs from data with a numeric-symbolic hybrid deep network. *J Comput Phys* 2019;399:108925.
- [11] Liu Z, Cai W, Xu ZQJ. Multi-scale deep neural network (MscaledNN) for solving Poisson–Boltzmann equation in complex domains. *Commun Comput Phys* 2020;28:1970–2001.
- [12] Pang G, Lu L, Karniadakis GE. fPINNs: fractional physics-informed neural networks. *SIAM J Sci Comput* 2019;41:A2603–26.
- [13] Jagtap AD, Kharazmi E, Karniadakis GE. Conservative physics-informed neural networks on discrete domains for conservation laws: applications to forward and inverse problems. *Comput Methods Appl Mech Engrg* 2020;365:113028.
- [14] Pang G, D’Elia M, Parks M, Karniadakis GE. nPINNs: nonlocal physics-informed neural networks for a parametrized nonlocal universal Laplacian operator. *Algorithms Appl* 2020;422:109760.
- [15] Zhang D, Guo L, Karniadakis GE. Learning in modal space: solving time-dependent stochastic PDEs using physics-informed neural networks. *SIAM J Sci Comput* 2020;42:639–65.
- [16] Sirignano J, Spiliopoulos K. DGM: a deep learning algorithm for solving partial differential equations. *J Comput Phys* 2018;375:1339–64.
- [17] Lyu L, Zhang Z, Chen M, Chen J. MIM: a deep mixed residual method for solving high-order partial differential equations. *J Comput Phys* 2022;110930.
- [18] Cai Z, Chen J, Liu M. Least-squares ReLU neural network (LSNN) method for linear advection-reaction equation. *J Comput Phys* 2021;110514.
- [19] Cai Z, Chen J, Liu M, Liu X. Deep least-squares methods: an unsupervised learning-based numerical method for solving elliptic PDEs. *J Comput Phys* 2020;420:109707.
- [20] Liao Y, Ming P. Deep Nitsche method: deep Ritz method with essential boundary conditions. *Commun Comput Phys* 2021;29:1365–84.
- [21] Sheng H, Yang C. PFNN: a penalty-free neural network method for solving a class of second-order boundary-value problems on complex geometries. *J Comput Phys* 2021;428:110085.
- [22] Zang Y, Bao G, Ye X, Zhou H. Weak adversarial networks for high-dimensional partial differential equations. *J Comput Phys* 2020;411:109409.
- [23] Kharazmi E, Zhang Z, Karniadakis GE. Variational physics-informed neural networks for solving partial differential equations. 2019, arXiv preprint arXiv:1912.00873.
- [24] Khodayi-Mehr R, Zavlanos M. VarNet: Variational neural networks for the solution of partial differential equations. In: *Learning for dynamics and control*. PMLR; 2020, p. 298–307.
- [25] Yang M, Foster JT. hp-Variational physics-informed neural networks for nonlinear two-phase transport in porous media. *J Mach Learn Model Comput* 2021;2.
- [26] Berrone S, Canuto C, Pintore M. Variational physics informed neural networks: the role of quadratures and test functions. *J Sci Comput* 2022;92:100.
- [27] Dong S, Li Z. Local extreme learning machines and domain decomposition for solving linear and nonlinear partial differential equations. *Comput Methods Appl Mech Engrg* 2021;387:114129.
- [28] Chen J, Chi X, E W, Yang Z. Bridging traditional and machine learning-based algorithms for solving PDEs: The random feature method. 2022, arXiv Preprint arXiv:2207.13380.
- [29] Cyr EC, Gulian MA, Patel RG, et al. Robust training and initialization of deep neural networks: An adaptive basis viewpoint. In: *Mathematical and scientific machine learning*. PMLR; 2020, p. 512–36.
- [30] Babuška I, Aziz AK. Lectures on the mathematical foundations of the finite element method. In: *Proceedings of a symposium*. University of Maryland; 1972.
- [31] Babuška I. The finite element method with Lagrangian multipliers. *Numer Math* 1973;20:179–92.
- [32] Boffi D, Brezzi F, Fortin M. Mixed finite element methods and applications. Springer series in computational mathematics, vol. 44, Heidelberg: Springer; 2013.
- [33] Xu J, Zikatanov L. Some observations on Babuška and Brezzi theories. *Numer Math* 2003;94:195–202.
- [34] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, p. 770–8.
- [35] Cybenko G. Approximation by superpositions of a sigmoidal function. *Math Control Signals Systems* 1989;2:303–14.
- [36] Hornik K. Approximation capabilities of multilayer feedforward networks. *Neural Netw* 1991;4:251–7.
- [37] Barron AR. Approximation and estimation bounds for artificial neural networks. *Mach Learn* 1994;14:115–33.
- [38] Mhaskar HN. Neural networks for optimal approximation of smooth and analytic functions. *Neural Comput* 1996;8:164–77.
- [39] Bolcskei H, Grohs P, Kutyniok G, Petersen P. Optimal approximation with sparsely connected deep neural networks. *SIAM J Math Data Sci* 2019;1:8–45.
- [40] Yarotsky D. Error bounds for approximations with deep ReLU networks. *Neural Netw* 2017;94:103–14.
- [41] Ohn I, Kim Y. Smooth function approximation by deep neural networks with general activation functions. *Entropy* 2019;21:627.
- [42] Gühring I, Raslan M. Approximation rates for neural networks with encodable weights in smoothness spaces. *Neural Netw* 2021;134:107–30.
- [43] Jiao Y, Lai Y, Lo Y, et al. Error analysis of deep Ritz methods for elliptic equations. 2021, arXiv Preprint arXiv:2107.14478.
- [44] Pao YH, Park GH, Sobajic DJ. Learning and generalization characteristics of the random vector functional-link net. *Neurocomputing* 1994;6:163–80.
- [45] Igel'nik B, Pao YH. Stochastic choice of basis functions in adaptive function approximation and the functional-link net. *IEEE Trans Neural Netw* 1995;6:1320–9.
- [46] Huang GB, Zhu QY, Siew CK. Extreme learning machine: theory and applications. *Neurocomputing* 2006;70:489–501.
- [47] Liu X, Lin S, Fang J, Xu Z. Is extreme learning machine feasible? A theoretical assessment (part 1). *IEEE Trans Neural Netw Learn Syst* 2014;26:7–20.
- [48] Raviart PA, Thomas JM. A mixed finite element method for 2-nd order elliptic problems. In: *Mathematical aspects of finite element methods*. Springer. 1977, p. 292–315.

- [49] He K, Zhang X, Ren S, Sun J. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In: Proceedings of the IEEE international conference on computer vision. 2015, p. 1026–34.
- [50] Glorot X, Bengio Y. Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of the thirteenth international conference on artificial intelligence and statistics. JMLR workshop and conference proceedings, 2010, p. 249–56.
- [51] Langtangena HP, Logg A. Solving PDEs in Python: The FEniCS tutorial I. Springer Nature; 2017.
- [52] Mojtabi A, Deville MO. One-dimensional linear advection–diffusion equation: Analytical and finite element solutions. *Comput & Fluids* 2015;107:189–95.
- [53] Mishra S, Molinaro R. Estimates on the generalization error of physics-informed neural networks for approximating a class of inverse problems for PDEs. *IMA J Numer Anal* 2022;42:981–1022.